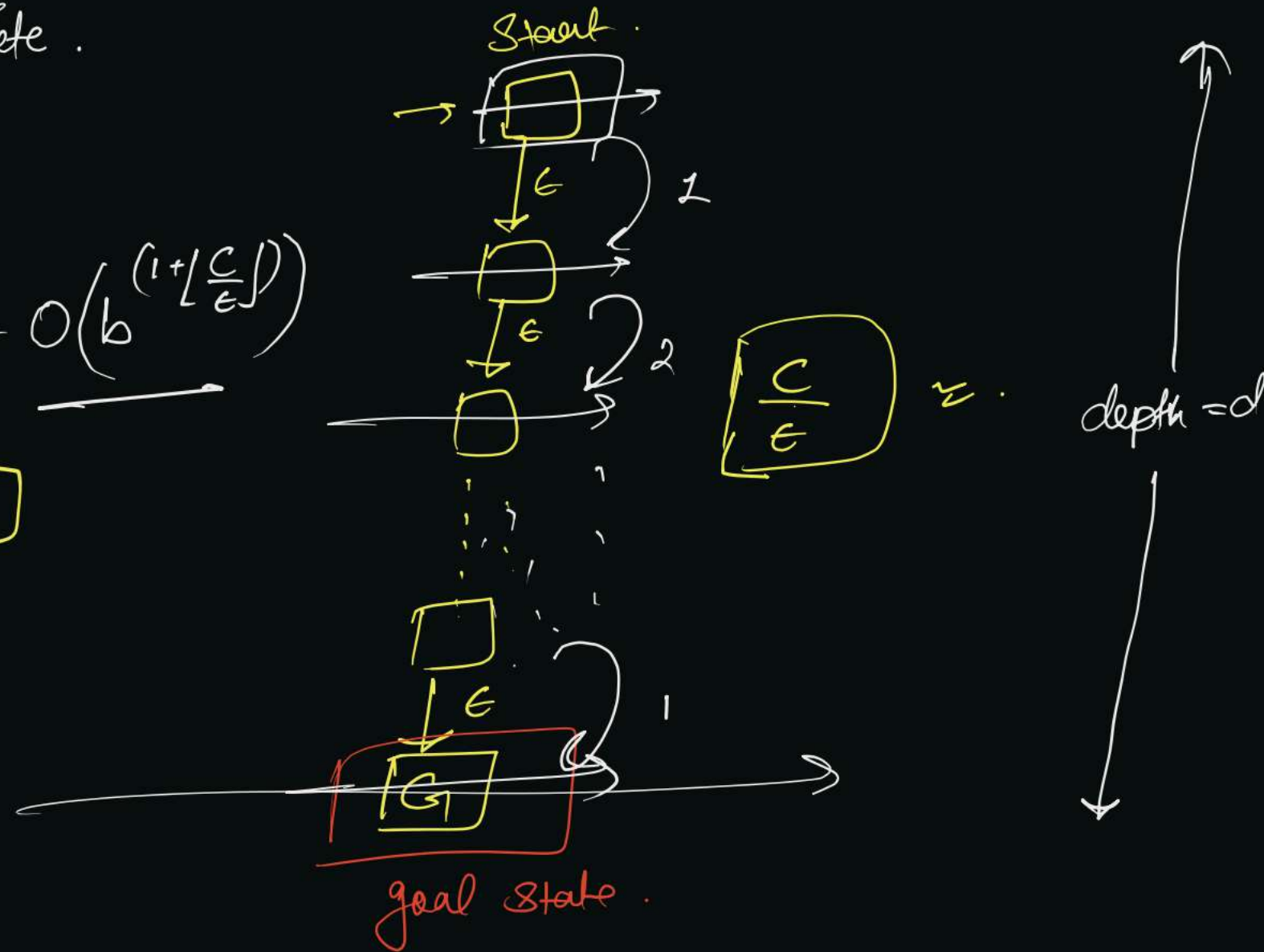


2. Time :-

time complexity = $O(b^d) = O(b^{(1+\frac{C}{\epsilon})})$

total optimal cost = C
min cost edge = ϵ

$$d = 1 + \left\lfloor \frac{c}{e} \right\rfloor$$



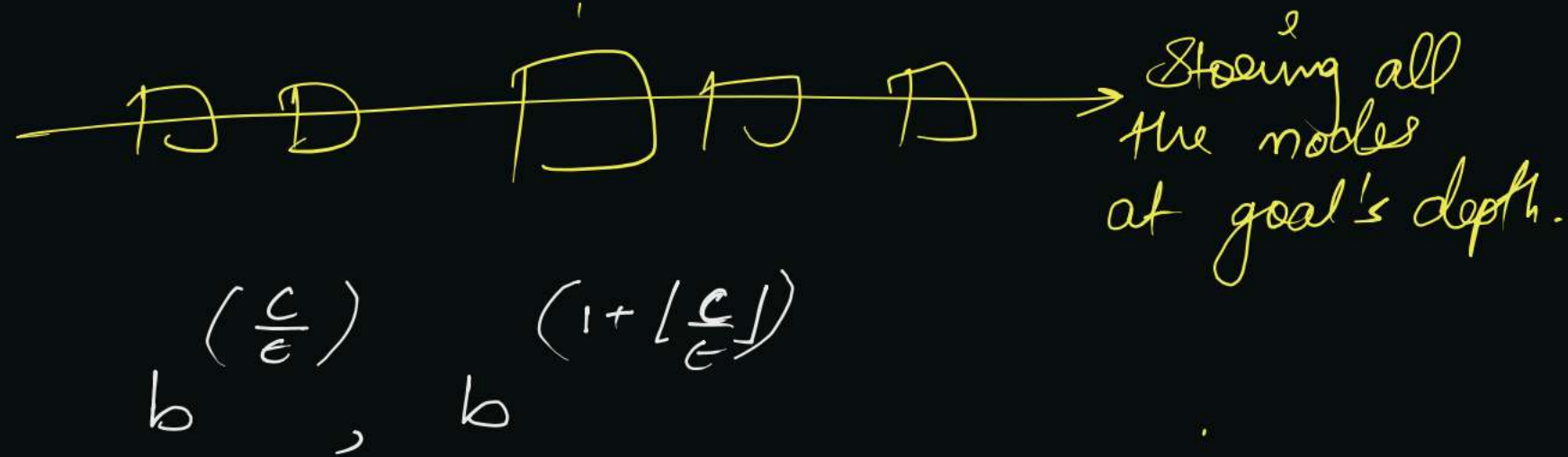
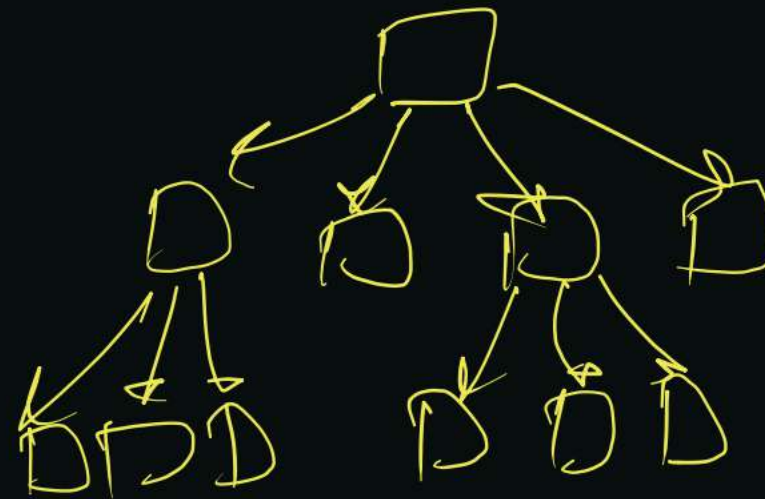
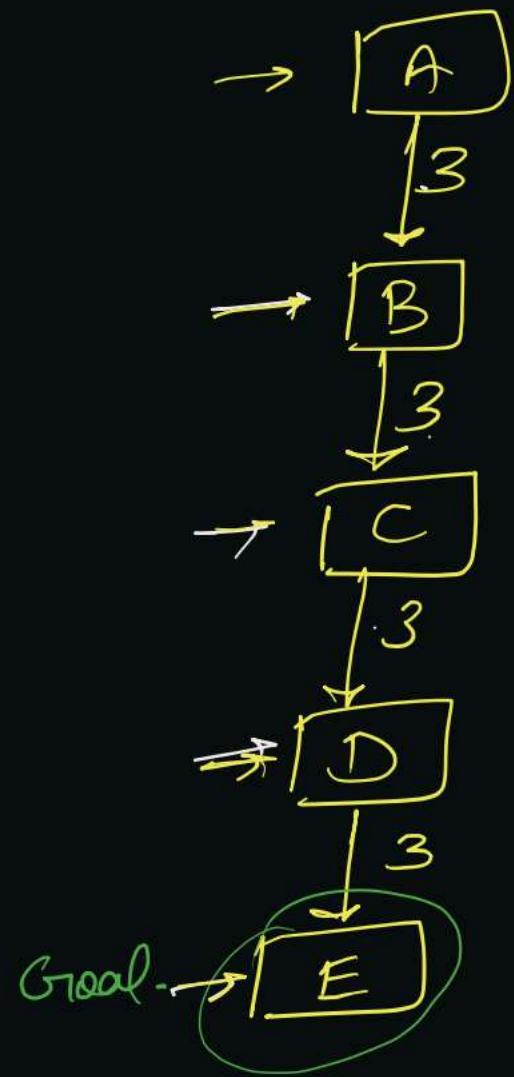
eg. $C = 12$, $\epsilon = 3$

$d = 4$

$$1 + \left\lceil \frac{C}{\epsilon} \right\rceil = 1 + \left\lceil \frac{12}{3} \right\rceil = 5$$

Hops = $\frac{C}{\epsilon} = 4$

$1 + \frac{C}{\epsilon} = 5$



3. Space :-

space complexity = $O(b^d) = O(b^{(1 + \lceil \frac{C}{\epsilon} \rceil)})$

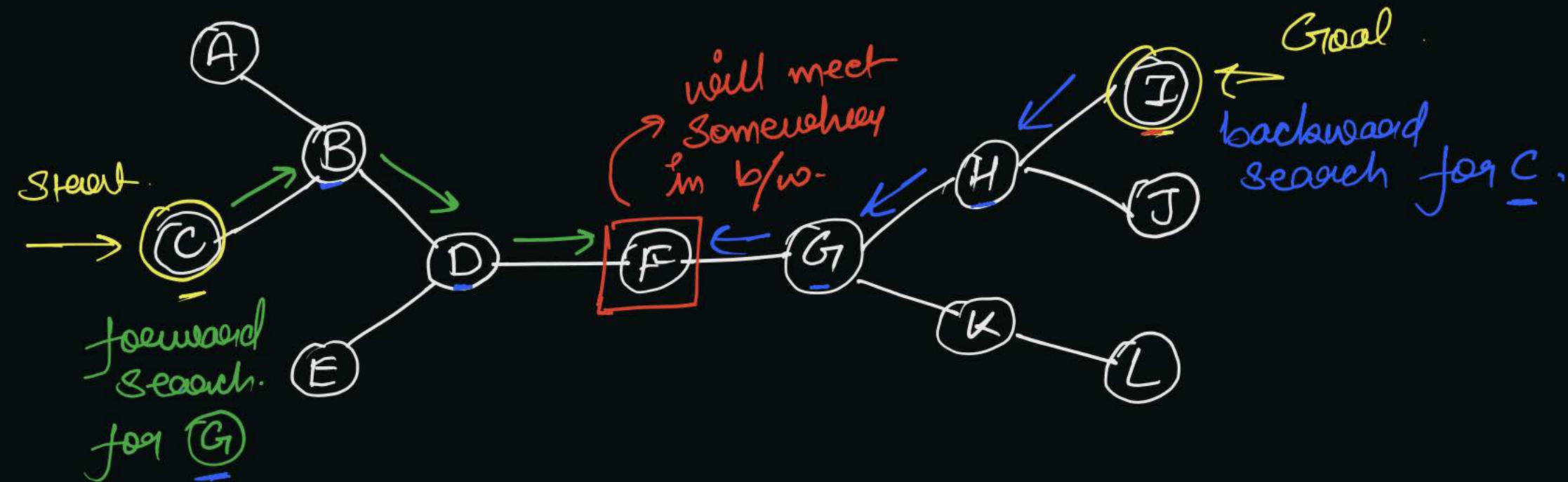
4. Optimal :- optimal (no -ve edge cycles).

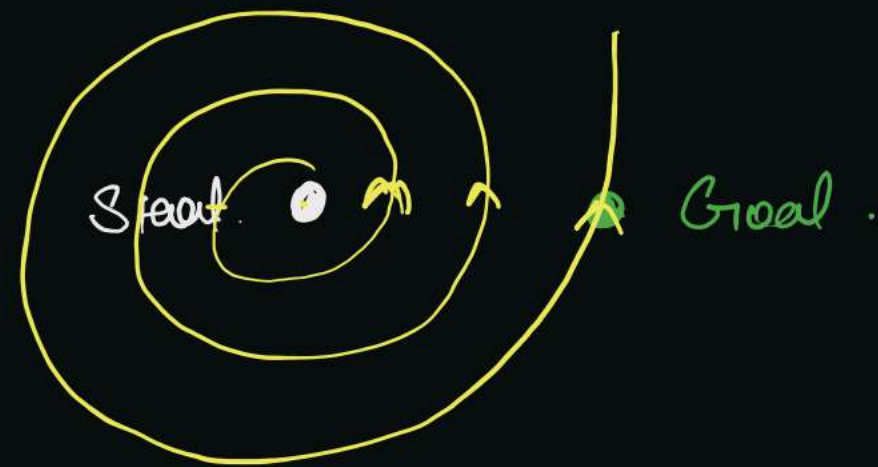
7. Bidirectional Search

Thursday, 19 September 2024 9:14 PM

Bidirectional Search :-

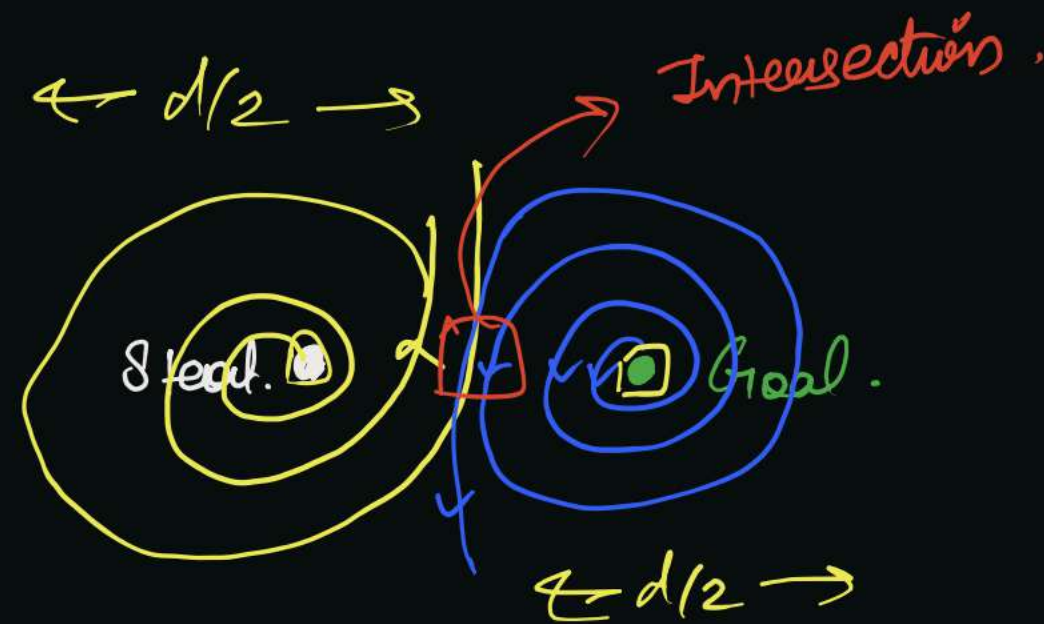
- ↳ Start searching from the start node → forward search.
- ↳ Parallelly start searching from the goal state → backward search.
- ↳ Hoping that the two searches will meet.





BFS .

$$O(b^d) \text{ \& \& } O(b^d)$$



Bidirectional -

$$\left. \begin{aligned} O(b^{d/2} + b^{d/2}) &= O(b^{d/2}) \\ O(b^{d/2} + b^{d/2}) &= O(b^{d/2}) \end{aligned} \right\}$$

Pseudo algorithm :-

take q_1, q_2 as the empty queues.

q_1 .enqueue (start)
marks start as visited.

q_2 .enqueue (goal)
marks goal as visited.

while (q_1 is not empty & q_2 is not empty)

{

$t_1 = q_1$.dequeue()

$t_2 = q_2$.dequeue()

if (q_1 is not empty)

{

for all neighbours t_1' of t_1

{

if ($t_1' \equiv g$ or $t_1' \in q_2$) $\rightarrow$ return t_1' ;

if (t_1' is not visited)

{

q_1 .enqueue (t_1') ;

marks t_1' as visited.

}

}

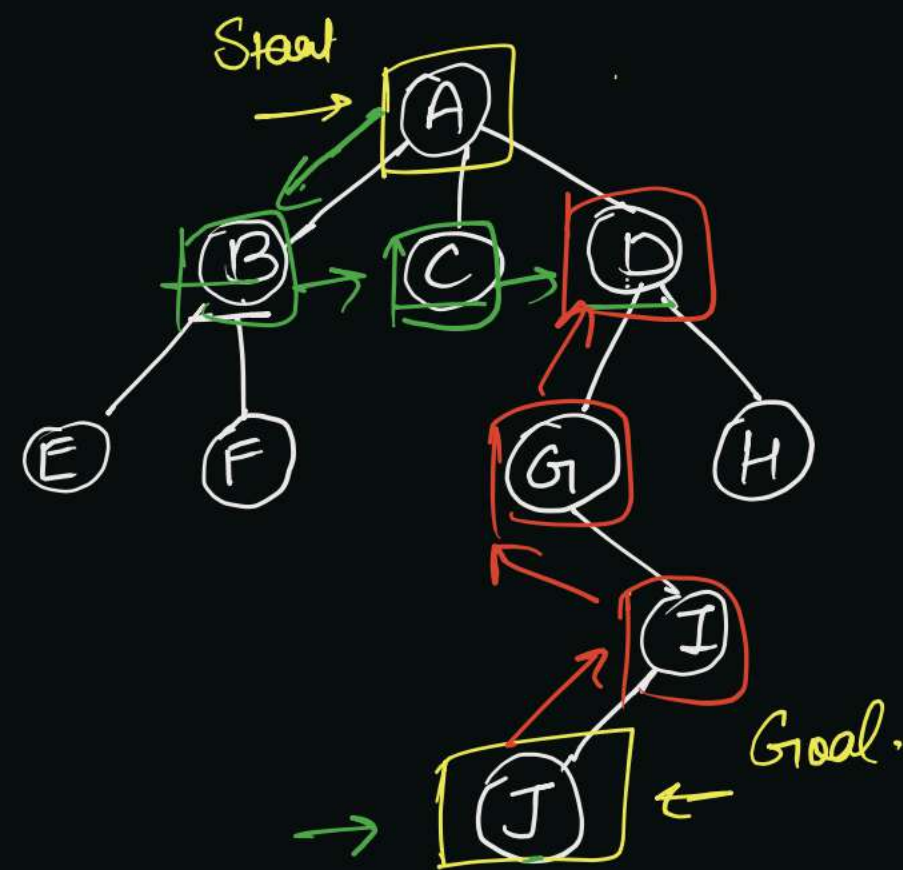
}

```

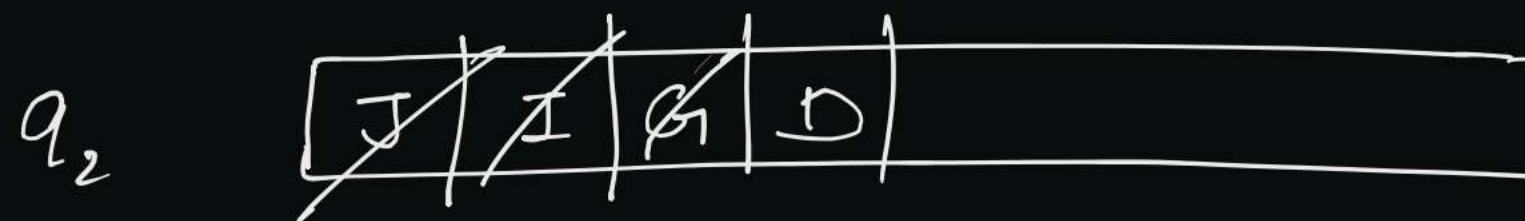
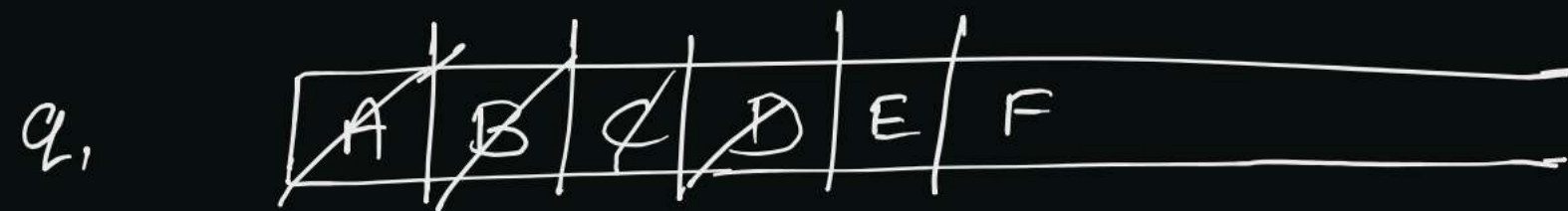
if ( $q_2$  is not empty)
{
  for all neighbours  $t_2'$  of  $t_2$ 
  {
    if ( $t_2' \in S$  or  $t_2' \in q_1$ )  $\rightarrow$  return  $t_2'$ ;
    if ( $t_2'$  is not visited)
    {
       $q_2$ .enqueue( $t_2'$ );
      marks  $t_2'$  as visited;
    }
  }
}
}
}

```

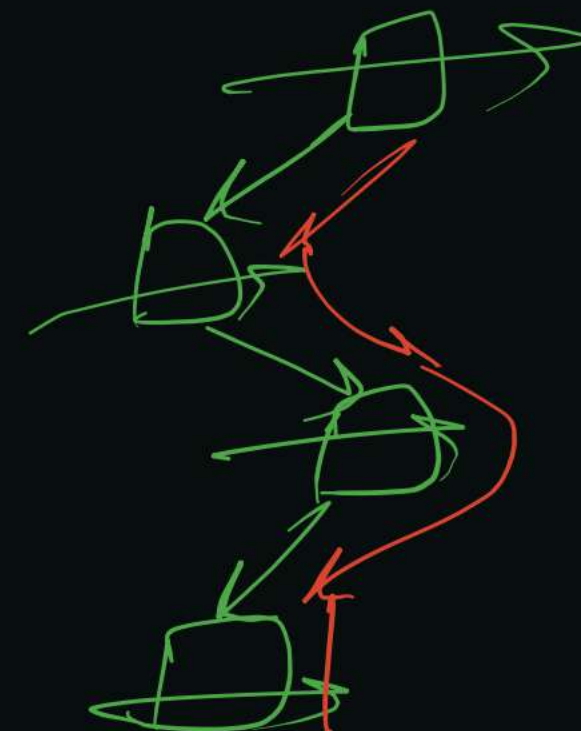
eg.



last on uninformed.



A	B	C	D	E	F	G	H	I	J
0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1		1	1



bfs $\equiv$ dfs

$\rightarrow t_1 = A$

$t_2 = J$

$t_1 = B$

$t_2 = I$

$t_1 = C$

$t_2 = G$

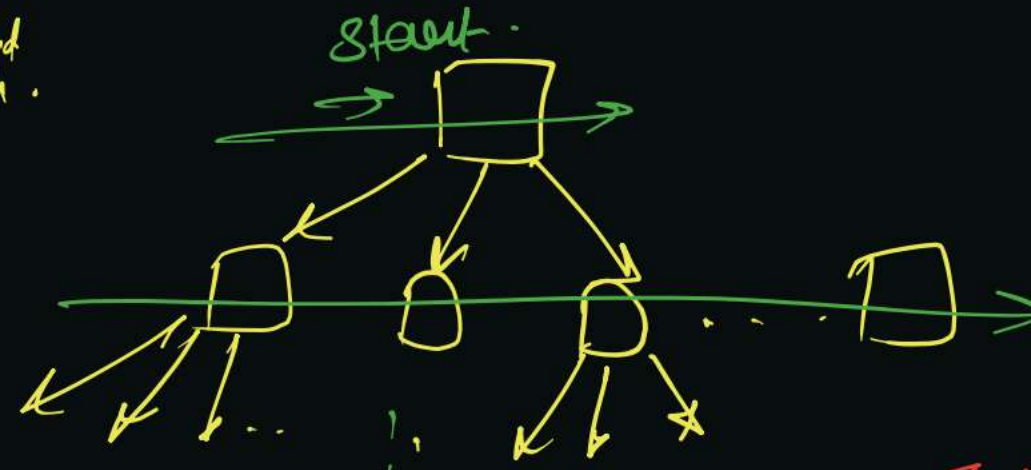
$t_1 = \textcircled{D} \rightarrow$

Performance analysis :-

1. Complete :- Yes, it will give you the solution (if exists).

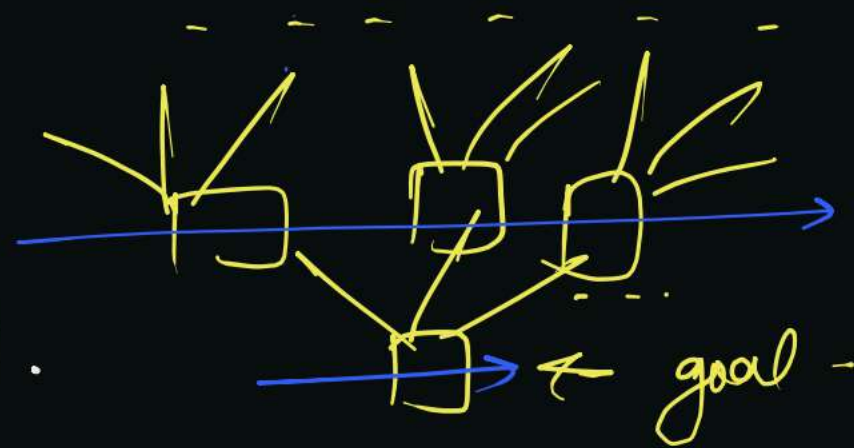
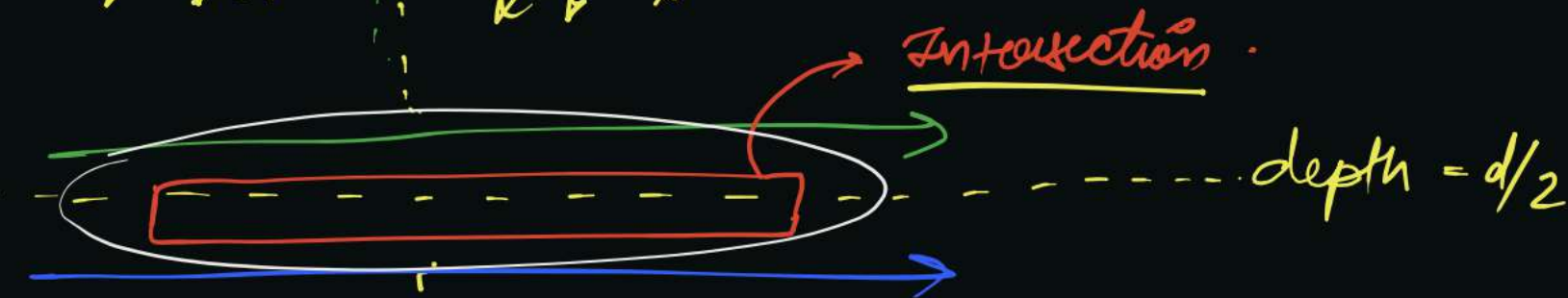
2. Time :-

forward search. backward search.

$$= O(b^{d/2}) + O(b^{d/2})$$
$$= O(b^{d/2})$$


3. Space :-

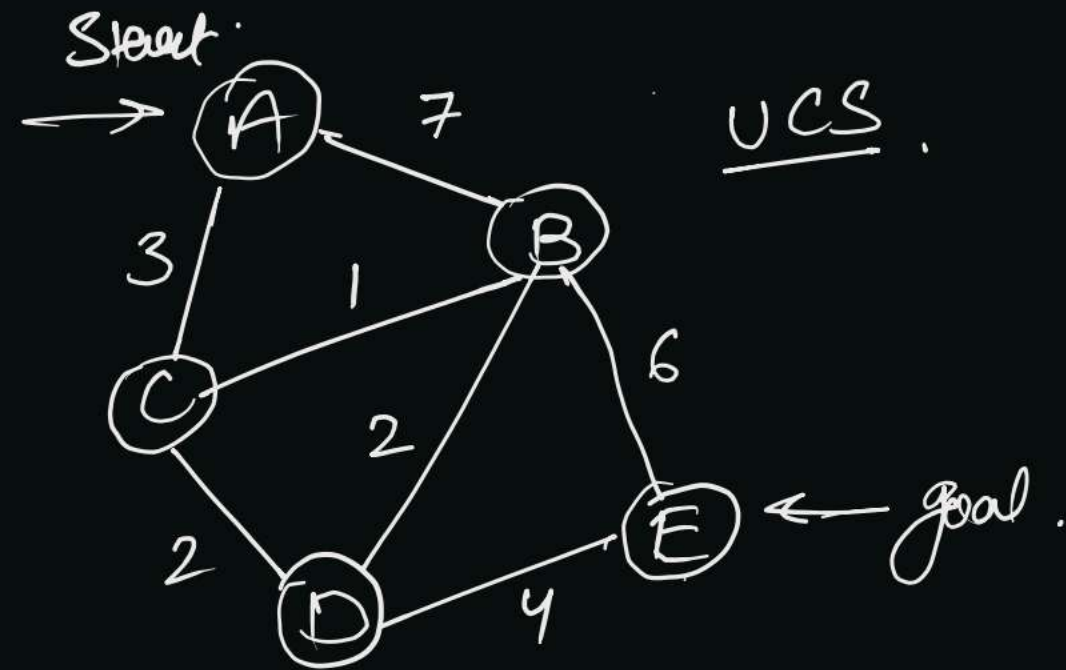
max. memory that forward search will take.

$$= O(b^{d/2}) + O(b^{d/2})$$
$$= O(b^{d/2})$$


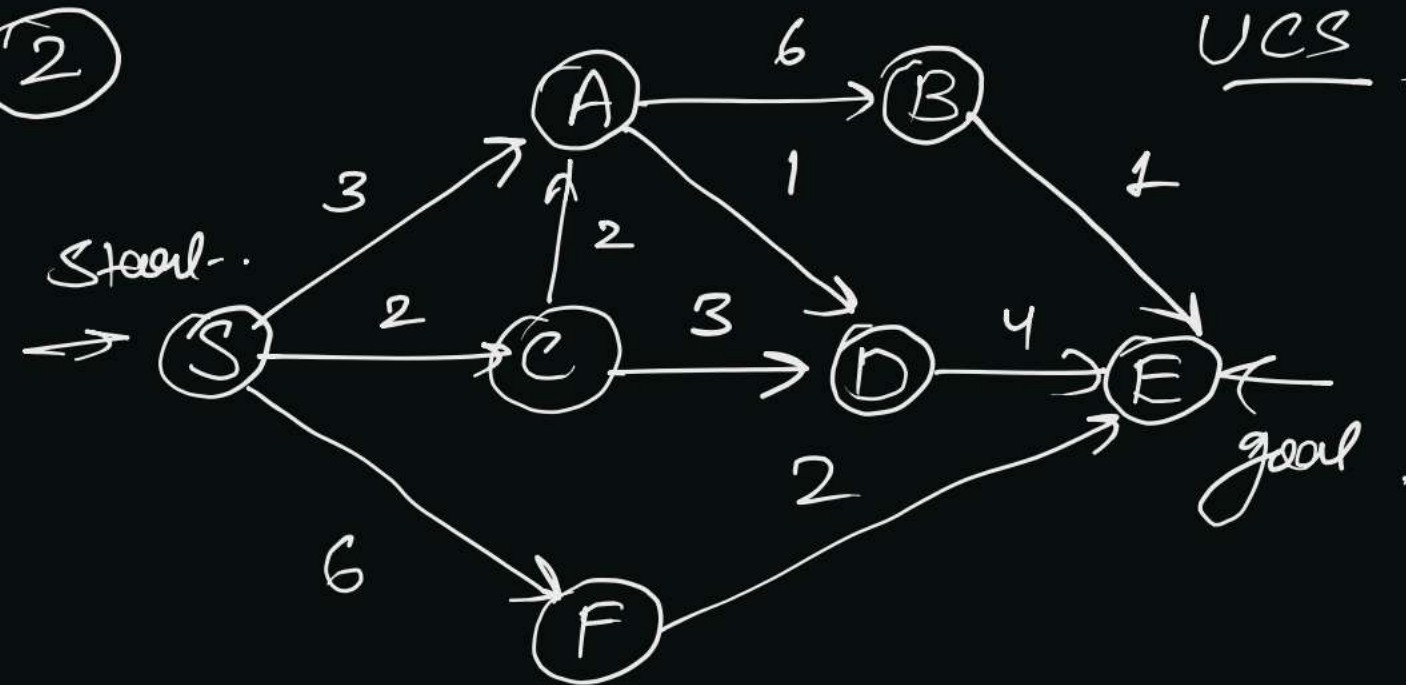
4. Optimal :- Yes (bfs is optimal).

Hw

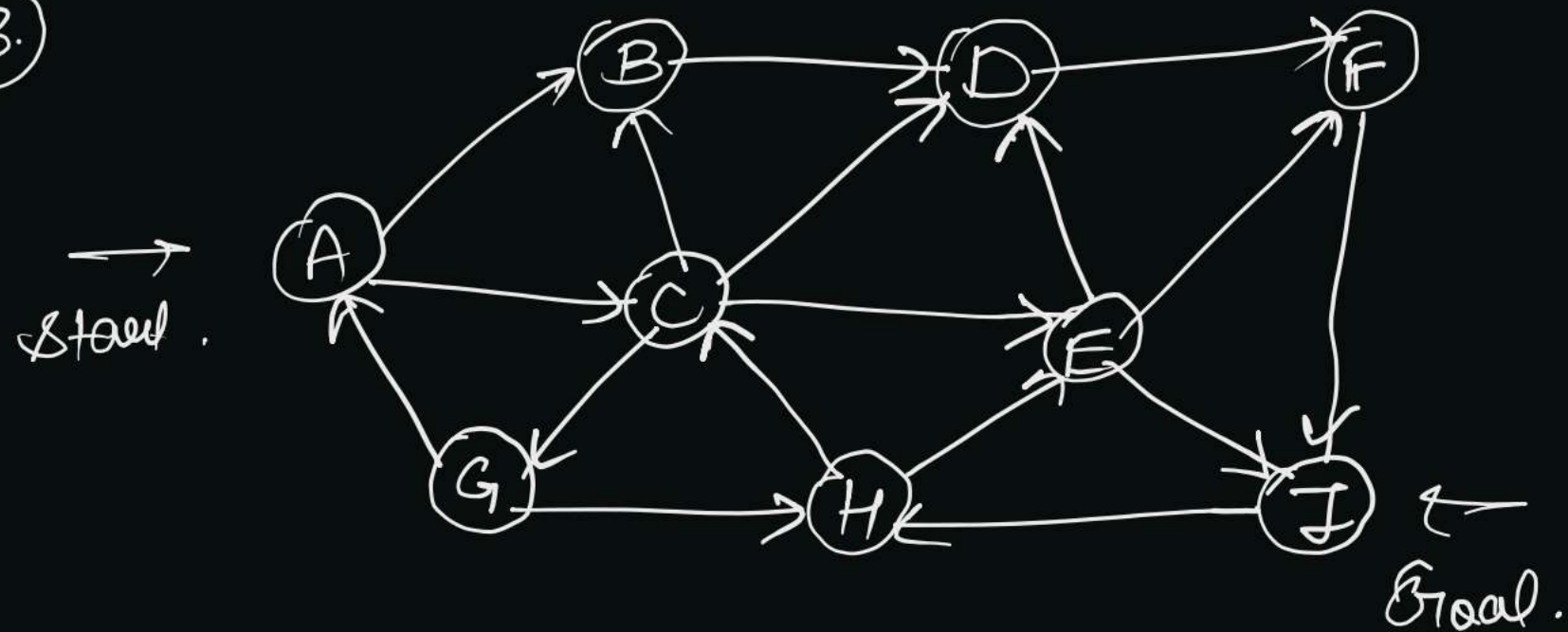
(1)



(2)



(3)



BFS, DFS, Bidirectional.

{ BFS, DFS, depth-limited, iterative deepening, Uniform Cost
Bidirectional.